

Project Proposal – Empirical Analysis of Saddle-Point Escaping Optimizers on Non-Convex ML Loss Landscapes

Shyam Patadia spatadia@wpi.edu
MA 551 Computational Statistics

1 Introduction

Training a machine learning model is fundamentally an optimization problem: minimize a loss function $f(\theta)$ over a high-dimensional, non-convex parameter space. In practice, neural network loss landscapes are riddled with saddle points – critical points where the gradient vanishes but the point is neither a minimum nor a maximum – and flat plateau regions where gradients are near-zero and learning stalls. In high dimensions, saddle points vastly outnumber local minima, making them the dominant obstacle in deep learning optimization.

A recent algorithm, Steepest Perturbed Gradient Descent (SPGD) [1], proposes a principled escape mechanism: every $IterP$ iterations, generate N_P uniformly perturbed candidate solutions around the current point and select the one yielding the steepest loss decrease. This combines the directed efficiency of gradient descent with a Monte Carlo exploration strategy. However, SPGD has only been evaluated on mathematical benchmark functions and a 3D engineering problem; its behavior on ML loss landscapes is entirely unexplored. This project provides the first systematic empirical study of SPGD on ML tasks, comparing it against established optimizers under controlled conditions.

2 Problem Description

Standard optimizers – SGD, Adam, and their variants – rely solely on gradient information. Near saddle points, gradients approach zero, causing these methods to stall for many iterations with negligible progress. Perturbed Gradient Descent (PGD) [2] adds random noise to escape such regions but treats all perturbation directions equally. SPGD’s steepest-selection rule is more principled: it evaluates N_P candidates and selects the most promising one. Whether this selection strategy provides a meaningful advantage over simpler perturbation approaches – and under what conditions – is an open empirical question with direct implications for ML practitioners choosing optimizers for non-convex problems.

3 Objectives

1. Implement SPGD as a PyTorch `Optimizer` subclass and validate it on non-convex benchmark functions (Rastrigin, Ackley, Rosenbrock) at $d \in \{2, 10, 50\}$.
2. Compare SPGD against SGD, Adam, PGD, and a random-perturbation baseline across four datasets spanning synthetic, tabular, and vision settings.
3. Diagnose saddle-point behavior through gradient-norm stagnation episodes and escape-time measurements to determine when and why SPGD outperforms competing methods.
4. Characterize SPGD’s sensitivity to N_P and $IterP$ via ablation on the Two Moons dataset.

4 Datasets and Experiments

Dataset	Type	Size	What it isolates	Est. compute
Rastrigin / Ackley / Rosenbrock	Synthetic	Generated	Benchmark validation, visual trajectories	<1 min
Two Moons (<code>sklearn</code>)	Synthetic 2D	1,000	Visual saddle escape, loss landscape plot	<5 min
OpenML-CC18 tabular subset	Real tabular	500–50k	Real-world generalization, 5 datasets	<20 min
CIFAR-10 (<code>torchvision</code>)	Real images	60,000	Scale stress test, practical convergence	~2 hrs

Experiment 1 – Benchmark Function Validation (Week 1)

SPGD is validated on Rastrigin ($f(x) = An + \sum[x_i^2 - A \cos(2\pi x_i)]$), Ackley, and Rosenbrock functions at $d \in \{2, 10, 50\}$ with 30 random initializations per optimizer. These functions have known global optima and well-characterized saddle/local-minima structure. Metrics: final function value, iterations to convergence, and success rate (fraction of runs reaching within $\epsilon = 0.01$ of the global optimum). 2D trajectory plots visualize SPGD escaping local minima that trap vanilla GD.

Experiment 2 – Two Moons: Visual Saddle Analysis (Week 2)

A small MLP ($2 \rightarrow 32 \rightarrow 32 \rightarrow 1$, no batch normalization) is trained on Two Moons for 2,000 steps across all five optimizers with 5 random seeds. The loss landscape is visualized by sweeping two weight dimensions and plotting the surface alongside optimizer trajectories. Stagnation episodes – consecutive steps where $\|\nabla f\| < 10^{-4}$ – are counted per optimizer. This is the primary visual experiment, producing the most interpretable figure in the paper.

Experiment 3 – OpenML Tabular: Real-Data Generalization (Week 2)

Five tabular classification datasets are drawn from the OpenML-CC18 benchmark suite (`pip install openml`), selected with $n \in [500, 50,000]$ rows and $p \in [10, 100]$ features. A 3-layer MLP is trained on each dataset for all five optimizers across 3 seeds. Metrics: training loss convergence, stagnation episode count, and test accuracy. This experiment tests whether SPGD’s advantage observed on synthetic problems transfers to real heterogeneous tabular data.

Experiment 4 – CIFAR-10: Scale Stress Test (Weeks 3–4)

A ResNet-18 (11M parameters, no batch normalization in early layers to preserve saddle structure) is trained on CIFAR-10 for 50 epochs across all five optimizers with 3 seeds. This is the anchor experiment. Metrics: training loss curve, gradient norm over time, stagnation episode count, final test accuracy, and sharpness at convergence (loss increase under small weight perturbation $\delta = 0.01$, following the flat-minima hypothesis). Total GPU time: approximately 2 hours on an RTX 4050.

Experiment 5 – Ablation on N_P and $IterP$ (Week 4)

A 4×4 grid over $N_P \in \{1, 5, 10, 20\}$ and $IterP \in \{5, 10, 50, 100\}$ is run on Two Moons (16 combinations $\times$ 5 seeds = 80 runs, each under 5 seconds). The result is a heatmap of final test accuracy and stagnation count, identifying the practical operating regime for SPGD.

5 Approach and Methodology

Implementation. SPGD is implemented as a PyTorch `Optimizer` subclass. Every $IterP$ steps, N_P candidates are sampled by adding uniform perturbations $\delta \sim \text{Uniform}(-r, r)^d$ to current parameters. The candidate with the largest loss decrease is accepted; otherwise the standard gradient step is taken. The random-perturbation baseline uses the same mechanism but selects the candidate randomly rather than by steepest descent, isolating the contribution of SPGD’s selection rule.

Diagnostics. Stagnation episodes (steps with $\|\nabla f\| < \varepsilon$) and escape times are logged at zero cost inside the training loop. These diagnostics directly measure saddle-point behavior rather than inferring it from loss curves alone.

Reproducibility. All experiments use fixed random seeds. Total compute across all experiments is under 3 hours on an RTX 4050.

References

- [1] A. M. Vahedi and H. T. Ilies, “SPGD: Steepest Perturbed Gradient Descent Optimization,” *arXiv:2411.04946*, 2024.
- [2] C. Jin, R. Ge, P. Netrapalli, S. M. Kakade, and M. I. Jordan, “How to Escape Saddle Points Efficiently,” *ICML*, 2017.